



UniSENAC — Centro Universitário SENAC

Análise e Desenvolvimento de Sistemas

Monitor de XML

Sistema automatizado de validação e monitoramento de erros em XMLs

Aluno: João Antonio Acunha Nogueira

Professor: Wagner Loch

Junho de 2026

Resumo

Este relatório apresenta o desenvolvimento do sistema Monitor de XML, uma solução para validação e monitoramento automático de arquivos XML gerados por sistemas de ponto de venda (PDV). O sistema monitora uma pasta em tempo real, valida cada arquivo XML detectando erros de estrutura e conteúdo, envia alertas automáticos por webhook (n8n) ou e-mail direto (fallback SMTP) quando um arquivo inválido é encontrado, e exporta relatórios de erros para análise em ferramentas de Business Intelligence. O monitor opera em silêncio, sem interface gráfica, iniciando automaticamente com o Windows.

Palavras-chave: *XML, XSD, Validação, PDV, NF-e, Automação, Python, n8n, Watchdog.*

1. Introdução

Em sistemas de ponto de venda (PDV), os arquivos XML são o principal meio de comunicação entre o software de frente de caixa, a SEFAZ e os sistemas de retaguarda. Documentos fiscais como a NF-e (Nota Fiscal Eletrônica) e o NCF-e (Nota Fiscal de Consumidor Eletrônica) são transmitidos obrigatoriamente nesse formato, seguindo esquemas XSD definidos pela legislação brasileira. Qualquer erro no XML — desde uma tag mal fechada até um campo obrigatório ausente — pode causar a rejeição do documento pela SEFAZ, interromper o fluxo de vendas e gerar inconsistências nos sistemas de estoque e financeiro.

Na prática, operadores de PDV frequentemente enfrentam erros de XML sem um mecanismo eficiente para detectá-los, identificar a causa e notificar a equipe técnica responsável. Os erros acabam sendo descobertos tardiamente — quando a NF-e já foi rejeitada ou quando o relatório de fechamento do caixa apresenta divergências.

O sistema desenvolvido neste trabalho resolve esse problema automatizando todo o ciclo de detecção de erros: monitora a pasta onde os XMLs são gerados, valida cada arquivo ao instante em que ele aparece, identifica e descreve os erros encontrados e envia um alerta imediato para a equipe técnica — tudo sem intervenção manual.

Os objetivos do projeto são:

- Detectar automaticamente novos arquivos XML na pasta de saída do PDV;
- Validar a estrutura e o conteúdo do XML (bem-formação e conformidade com XSD);
- Enviar alerta automático com a descrição dos erros via webhook (n8n) ou e-mail direto (fallback SMTP);
- Arquivar os XMLs processados e gerar log de erros para auditoria;
- Exportar relatório de erros em CSV para análise em Power BI ou Excel;
- Operar em silêncio, sem interface gráfica, iniciando automaticamente com o Windows.

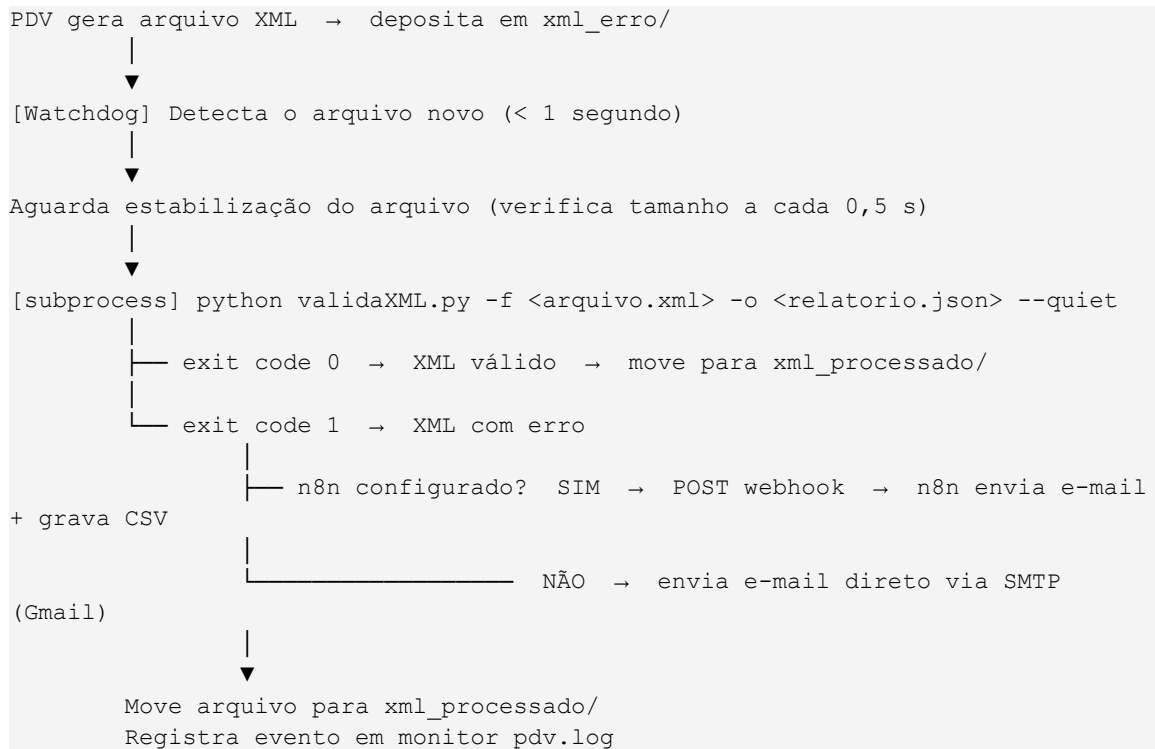
2. Arquitetura do Sistema

O sistema é dividido em quatro camadas funcionais independentes. Cada camada tem responsabilidade única e se comunica com a seguinte por interfaces bem definidas (exit codes e arquivos JSON):

Camada	Módulo principal	Responsabilidade
Aquisição	monitor_pdv.py	Detecta novos XMLs na pasta de entrada em tempo real
Validação	validaXML.py	Analisa o XML e reporta erros de estrutura e conteúdo
Notificação	monitor_pdv.py	Envia alerta via webhook n8n ou e-mail (fallback)
Reporting	gerar_relatorio_bi.py	Exporta resultados para JSON e CSV

O monitor opera em segundo plano, sem janela visível. Em produção, é iniciado automaticamente no login do Windows via Windows Task Scheduler, usando o iniciar_silencioso.vbs. Todo o processamento é registado em monitor_pdv.log.

Fluxo completo para cada arquivo XML que entra no sistema:



3. Implementação

3.1. Motor de Validação — validaXML.py

O validaXML.py é o núcleo do sistema. Roda como processo separado chamado via subprocess, o que isola qualquer falha de parsing do processo monitor principal e permite aplicar um timeout rígido (padrão: 60 segundos). Suporta três tipos de validação:

- Bem-formação: verifica se o XML está sintaticamente correto — tags fechadas, caracteres especiais escapados, encoding declarado. Usa a biblioteca lxml (com fallback para ElementTree);
- XSD: valida o documento contra um esquema XSD, reportando cada violação com mensagem, linha e coluna;
- XPath: verifica se determinados elementos existem no documento através de expressões XPath.

Parâmetros aceitos na linha de comando:

Argumento	Descrição
-f / --file	Arquivo XML a validar
--xsd	Esquema XSD para validação de estrutura
--xpath	Expressões XPath para verificar presença de elementos
--batch	Pasta para validação em lote de múltiplos arquivos
-o / --output	Arquivo JSON de saída com o relatório detalhado
--quiet	Sem output no terminal — retorna apenas o exit code

Códigos de retorno (exit codes):

Código	Significado
0	XML válido — todas as validações passaram
1	XML inválido — pelo menos uma validação falhou
2	Erro de uso — arquivo não encontrado ou parâmetro inválido

3.2. Monitor de Diretório — monitor_pdv.py

O monitor usa a biblioteca Watchdog, que utiliza a API nativa do Windows (ReadDirectoryChangesW) para receber notificações do sistema operacional quando um arquivo é criado. A classe MonitorPDV herda de FileSystemEventHandler e responde ao evento on_created.

Dois problemas comuns no Windows foram tratados na implementação:

- Eventos duplicados: o Watchdog pode disparar on_created em threads paralelas para o mesmo arquivo. Resolvido com um threading.Lock e um conjunto de arquivos em processamento — a verificação e a adição são atômicas (check-and-add);

- Arquivo incompleto: o evento é disparado antes de o PDV terminar de escrever o arquivo. Resolvido monitorando o tamanho do arquivo a cada 0,5 s até estabilizar, com limite de 5 segundos.

O envio de alertas usa retry com backoff exponencial: após cada falha aguarda 2, 4 e 8 segundos antes da próxima tentativa (máximo 3). Se o webhook n8n não estiver configurado, o alerta é enviado diretamente por SMTP com o relatório JSON em anexo.

3.3. Interface Gráfica — validador_gui.py (uso opcional)

Como ferramenta auxiliar, o projeto inclui uma interface gráfica desenvolvida com Tkinter que permite validar arquivos XML manualmente sem o terminal. O fluxo principal do sistema — monitor, validação e alerta — funciona inteiramente em segundo plano, sem esta interface.

3.4. Relatório BI e Scripts de Automação

O módulo gerar_relatorio_bi.py converte o relatório JSON gerado pelo validador em lote para um CSV com nome datado (dashboard_bi_YYYYMMDD.csv), pronto para importar no Power BI ou Excel. As colunas geradas são: arquivo, valido, erros_quantidade e timestamp — permitindo analisar a evolução da taxa de erros por período e identificar padrões recorrentes.

Scripts de automação de inicialização:

Script	Função
instalar_autostart.bat	Registra a tarefa 'MonitorPDV' no Windows Task Scheduler (trigger: login, nível Admin)
desinstalar_autostart.bat	Remove a tarefa agendada
iniciar_silencioso.vbs	Inicia o monitor em janela completamente oculta via Windows Script Host — nenhuma janela visível ao utilizador
start_n8n.bat	Inicia o n8n manualmente com janela visível (para diagnóstico)
start_monitor.bat	Inicia o monitor manualmente com janela visível (para diagnóstico)

4. Caso de Uso — Validação de XML de PDV

Para demonstrar o funcionamento do sistema foram incluídos na pasta exemplos/ dois arquivos XML e um schema XSD. O arquivo venda_ok.xml representa uma venda corretamente gerada pelo PDV. O arquivo venda_erro.xml contém um erro proposital que representa uma falha comum encontrada em produção.

4.1. Schema XSD — exemplos/schema_pdv.xsd

O schema define a estrutura obrigatória de um registro de venda. Os campos numero, data, cliente, itens e total são todos obrigatórios, e quantidade, valor_unitario e valor_total devem ser do tipo xs:decimal:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="venda">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="numero" type="xs:integer"/>
        <xs:element name="data" type="xs:dateTime"/>
        <xs:element name="cliente">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="cpf" type="xs:string"/>
              <xs:element name="nome" type="xs:string"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
        <xs:element name="itens">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="item" maxOccurs="unbounded">
                <xs:complexType>
                  <xs:sequence>
                    <xs:element name="codigo" type="xs:string"/>
                    <xs:element name="descricao" type="xs:string"/>
                    <xs:element name="quantidade" type="xs:decimal"/>
                    <xs:element name="valor_unitario" type="xs:decimal"/>
                    <xs:element name="valor_total" type="xs:decimal"/>
                  </xs:sequence>
                </xs:complexType>
              </xs:element>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
        <xs:element name="total" type="xs:decimal"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

4.2. XML com Erro — exemplos/venda_erro.xml

O arquivo abaixo contém um erro de bem-formação: o caractere '&' na descrição do produto não foi escapado corretamente. Adicionalmente, o campo <quantidade> contém o texto 'tres' em vez de um número — o que causaria erro XSD após a correção do primeiro erro:

```
<?xml version="1.0" encoding="UTF-8"?>
<venda>
  <numero>1525</numero>
  <data>2026-06-17T18:10:00</data>
  <cliente>
    <cpf>987.654.321-00</cpf>
    <nome>Maria Santos</nome>
  </cliente>
  <itens>
    <item>
      <codigo>456789012</codigo>
      <descricao>AGUA MINERAL & NATURAL</descricao>    <!-- ERRO: & nao escapado
-->
      <quantidade>tres</quantidade>                    <!-- tipo errado:
xs:decimal esperado -->
      <valor_unitario>3.00</valor_unitario>
      <valor_total>9.00</valor_total>
    </item>
  </itens>
  <total>9.00</total>
</venda>
```

Erro reportado pelo validador (saída real do sistema):

```
Erro de sintaxe XML: xmlParseEntityRef: no name, line 12, column 32
(venda_erro.xml, line 12)
```

O parser rejeita o arquivo na linha 12 ao encontrar o '&' sem escape. O processamento para imediatamente — exit code 1 — e o monitor dispara o alerta antes de mover o arquivo para xml_processado/. Se o '&' fosse corrigido para '&#38;', a segunda validação (XSD) detectaria o erro de tipo no campo <quantidade>.

4.3. XML Válido — exemplos/venda_ok.xml

O arquivo abaixo passa em todas as validações (bem-formação e XSD) e recebe exit code 0:

```
<?xml version="1.0" encoding="UTF-8"?>
<venda>
  <numero>1524</numero>
  <data>2026-06-17T18:05:00</data>
  <cliente>
    <cpf>123.456.789-00</cpf>
    <nome>Joao Silva</nome>
  </cliente>
  <itens>
    <item>
      <codigo>789101010</codigo>
      <descricao>REFRIGERANTE COLA</descricao>
```

```
<quantidade>2</quantidade>  
<valor_unitario>7.50</valor_unitario>  
<valor_total>15.00</valor_total>  
</item>  
</itens>  
<total>15.00</total>  
</venda>
```

5. Integração com n8n — Exemplo Completo de Erro

O n8n é uma ferramenta de automação de workflows que roda localmente na porta 5678. Ele recebe os alertas do monitor via HTTP POST e os processa em um workflow com quatro nós. A seguir é apresentado o caminho completo percorrido pelo arquivo venda_erro.xml desde a detecção do erro até a notificação da equipe.

5.1. Estrutura do Workflow



5.2. Passo 1 — Monitor detecta o arquivo e chama o validador

Ao detectar venda_erro.xml na pasta xml_erro/, o monitor executa o validador em subprocess e recebe exit code 1 (inválido). Em seguida, monta o payload JSON e faz o POST para o n8n:

```
{
  "arquivo": "venda_erro.xml",
  "timestamp": "2026-06-17T18:05:42.381Z",
  "erros": [
    "xmlParseEntityRef: no name, line 12, column 32 (venda_erro.xml, line 12)"
  ],
  "relatorio": {
    "file": "venda_erro.xml",
    "is_valid": false,
    "errors": [ "xmlParseEntityRef: no name, line 12, column 32" ],
    "duration_ms": 3.8,
    "file_size_kb": 0.7,
    "timestamp": "2026-06-17T18:05:42.381Z"
  }
}
```

5.3. Passo 2 — Nó 'Formatar Dados' processa o payload

O nó de código JavaScript lê o payload recebido e prepara dois outputs: o corpo do e-mail em HTML e a linha que será gravada no CSV.

```
// Trecho do código JavaScript no nó 'Formatar Dados' do n8n

const arquivo = $input.item.json.arquivo;
const timestamp = $input.item.json.timestamp;
const erros = $input.item.json.erros;

// Monta lista de erros em HTML
```

```
const listaErros = erros.map(e => `<li>${e}</li>`).join('');

const emailHtml = `
  <h2 style="color:#c0392b">Erro em XML de PDV</h2>
  <p><b>Arquivo:</b> ${arquivo}</p>
  <p><b>Data/Hora:</b> ${timestamp}</p>
  <p><b>Erros encontrados (${erros.length}):</b></p>
  <ul>${listaErros}</ul>
`;

// Linha para o CSV
const csvLinha = `${timestamp},${arquivo},${erros.length}`;

return [{ json: { emailHtml, csvLinha, arquivo, erros } }];
```

5.4. Passo 3 — E-mail recebido pela equipe técnica

O nó 'Enviar E-mail' utiliza as credenciais Gmail configuradas no n8n e dispara a mensagem com o seguinte conteúdo:

```
Assunto: [ALERTA PDV] Erro em XML - venda_erro.xml
```

ERRO EM XML DE PDV

Arquivo : venda_erro.xml
Data/Hora: 2026-06-17T18:05:42.381Z

Erros encontrados (1):

- xmlParseEntityRef: no name, line 12, column 32
(caractere '&' não escapado em
<descricao>AGUA MINERAL & NATURAL</descricao>)

5.5. Passo 4 — Linha gravada no CSV de log

Simultaneamente ao e-mail, o nó 'Gravar Log CSV' acrescenta uma linha ao arquivo erros_pdv.csv, que serve de histórico para análise em Power BI ou Excel:

```
# erros_pdv.csv (cabeçalho + exemplo de linha gerada)

timestamp,arquivo,qtd_erros
2026-06-17T18:05:42.381Z,venda_erro.xml,1
```

Com o CSV acumulando registros ao longo do tempo, é possível identificar períodos com maior incidência de erros, quais arquivos são gerados com mais problemas e acompanhar a evolução da qualidade dos XMLs produzidos pelo PDV.

A principal vantagem de usar o n8n como camada de notificação é a extensibilidade: para adicionar um novo canal — Slack, Teams, WhatsApp — basta arrastar um nó novo no editor visual do n8n, sem alterar nenhuma linha do código Python. O fallback de e-mail direto via SMTP garante que o alerta chegue mesmo se o n8n estiver offline.

6. Testes

Foram realizados testes funcionais manuais cobrindo os principais cenários de uso. Todos os casos passaram com sucesso.

ID	Cenário testado	Resultado
T01	XML bem-formado, sem XSD	PASS — exit code 0
T02	XML com tag não fechada	PASS — exit code 1, erro de sintaxe
T03	XML com '&' não escapado	PASS — exit code 1, erro de caractere inválido
T04	XML com tag de fechamento errada (typo)	PASS — exit code 1, erro de bem-formação
T05	XML válido contra XSD	PASS — exit code 0
T06	XML com elemento obrigatório ausente no XSD	PASS — exit code 1, erro XSD com linha
T07	XML com tipo de dado errado no XSD	PASS — exit code 1, erro de tipo
T08	Processamento em lote — pasta mista	PASS — relatório JSON com resultado por arquivo
T09	Arquivo inexistente	PASS — exit code 2
T10	Detecção automática ao depositar XML na pasta	PASS — detectado em menos de 1 segundo
T11	Dois arquivos depositados em simultâneo	PASS — ambos processados, sem duplicatas
T12	Alerta com n8n ativo	PASS — webhook recebido, e-mail enviado
T13	Alerta com n8n offline (fallback SMTP)	PASS — e-mail enviado diretamente
T14	Inicialização automática no login do Windows	PASS — monitor e n8n ativos após reinício

7. Conclusão

O sistema Monitor de XML resolve um problema concreto do ambiente de ponto de venda: a detecção tardia de erros em arquivos XML. Com o sistema em operação, qualquer XML inválido que seja depositado na pasta de entrada é detectado em menos de um segundo, validado, e a equipe técnica recebe imediatamente um e-mail com o nome do arquivo, o timestamp e a descrição exata do erro — incluindo linha e coluna. O arquivo é então arquivado automaticamente. Todo este ciclo ocorre sem nenhuma intervenção humana e sem nenhuma janela aberta no computador.

O validador roda em subprocess isolado para que uma falha de parsing não interrompa o monitor. O `threading.Lock` com conjunto de arquivos em processamento elimina duplicatas mesmo sob carga. O `retry` com `backoff` exponencial absorve falhas de rede transitórias. E o fallback de SMTP garante que o alerta chegue mesmo que o n8n esteja indisponível — os dois caminhos foram testados e verificados em funcionamento.

A integração com o n8n acrescenta extensibilidade sem modificar o código Python: novos canais de notificação (Slack, Teams, WhatsApp) são adicionados arrastando um nó no editor visual. O CSV gerado em paralelo pelo workflow permite análise histórica da taxa de erros em Power BI ou Excel, transformando ocorrências individuais em inteligência operacional.

Como evoluções naturais do projeto identificam-se: validação contra os esquemas XSD oficiais da SEFAZ para NF-e e CF-e; uma dashboard web leve para visualização dos erros em tempo real; e cobertura de testes automáticos com `pytest` para garantir regressão zero em futuras alterações.

8. Referências

W3C (1998). Extensible Markup Language (XML) 1.0. W3C Recommendation. <https://www.w3.org/TR/xml/>

W3C (2001). XML Schema Part 0: Primer. W3C Recommendation. <https://www.w3.org/TR/xmlschema-0/>

W3C (1999). XML Path Language (XPath) Version 1.0. <https://www.w3.org/TR/xpath/>

Ministério da Fazenda — SEFAZ (2023). Manual de Orientação ao Contribuinte — NF-e, versão 7.0. Disponível em: <https://www.nfe.fazenda.gov.br/portal/>

Gamma, E., Helm, R., Johnson, R. & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Python Software Foundation. watchdog — Filesystem events monitoring. <https://python-watchdog.readthedocs.io/>

Python Software Foundation. lxml — XML and HTML processing library. <https://lxml.de/>

n8n GmbH. n8n Workflow Automation — Documentation. <https://docs.n8n.io/>